JAS: Einstein's Epistemology as a Running Architecture — Induction, Refutation, and the Warrant of Impossibility

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

JAS: Einstein's Epistemology as a Running Architecture — Induction, Refutation, and the Warrant of Impossibility **1**
Abstract . . . 1
1. The Diagram, and Why Its Asymmetry Matters . . . 1
1.1 What the substrate already had, and what was missing . . . 2
2. The Jump . . . 2
2.1 Reading the world, not being told about it . . . 2
2.2 Leaping further than the evidence . . . 3
3. Deduction and Experiment . . . 3
3.1 A -> S . . . 3
3.2 S -> E: the oracle . . . 3
3.3 The refutation . . . 3
4. The Cycle Turns: Revision and Re-Jump . . . 4
4.1 Testing on held-out symbols . . . 4
5. The Impossibility Register . . . 4
5.1 The problem . . . 4
5.2 Four warrants . . . 4
5.3 The canonical case . . . 5
5.4 The register, applied to this project's own claims . . . 5
5.5 The rule the substrate applies . . . 6
6. Why This Is Not Reinforcement Learning . . . 6
7. Falsifiability of This Paper's Own Claims . . . 6
8. Limitations . . . 7
9. Conclusion . . . 7
References . . . 7

JAS: Einstein's Epistemology as a Running Architecture — Induction, Refutation, and the Warrant of Impossibility

Irfan Mahir *Independent Researcher, Dhaka, Bangladesh* irfan@furylogic.com

> *"A system that cannot be wrong is not doing science. It is doing bookkeeping with a confident tone."*

Abstract

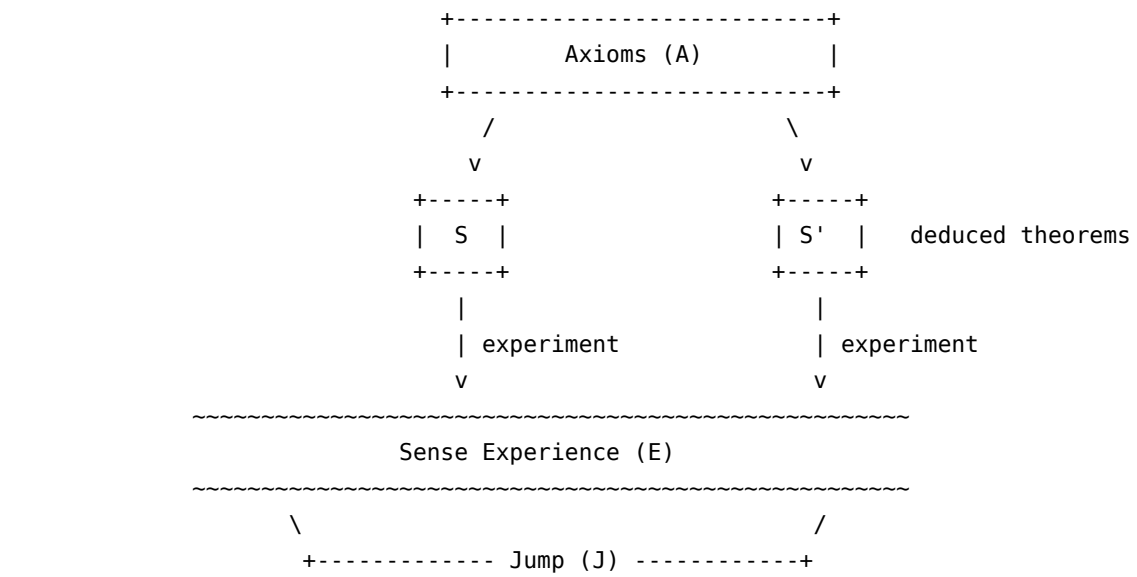
Learning systems are conventionally organised around an objective that is minimised. This paper describes an architecture organised instead around a claim that can be destroyed. We implement Einstein's account of physical theory — the **J-A-S cycle**, in which an unjustified *jump* (J) from sense experience produces axioms (A), from which theorems (S) are deduced and tested back against experience — as a running loop with an external oracle. Induction is counting; deduction is graph traversal; refutation is a compiler exit code. There is no loss function, no gradient, no fitted parameter, and no policy.

We report a deliberately false induction, its experimental refutation, and its revision into a restricted claim that survives on held-out symbols. We then introduce the **impossibility register**, which records the *warrant* of every "X cannot be done" claim in the system. The central argument is that warrant determines what a counterexample means: refuting an INDUCED universal ends it, while a counterexample to a THEOREM is a **scope error until proven otherwise**. Two of this project's own impossibility theorems were escaped by leaving their scope; neither was violated. We argue that this distinction — routinely collapsed in machine-learning discourse — is what separates a field that knows where its exits are from one that either stalls at a proof it misread or celebrates a breakthrough it did not achieve.

---

1. The Diagram, and Why Its Asymmetry Matters

Einstein described the structure of physical theory not as a ladder from observation to law, but as a cycle:



The load-bearing feature is that **J is not logical**. No inference rule carries you from a set of observations to an axiom. The arrow is inductive, unjustified, and revisable. Everything below the axioms is deductive and mechanical; everything from experience up to the axioms is a leap.

This asymmetry is what most learning architectures erase. Gradient descent presents itself as a continuous ascent from data to model, with no moment at which something unjustified is asserted. The claim of this paper is that erasing the leap does not remove it — it only removes the system's ability to notice that it made one, and therefore its ability to retract it.

1.1 What the substrate already had, and what was missing

Prior work in this lineage supplied every element of the diagram except the leap:

Element	Mechanism	Where
E — sense experience	mechanical AST extraction from source	`tools/extract_ast_axioms.py`
A — axioms	typed concept graph	`data/*.axioms`
S — theorems	forward chaining over nine primitives	100 derived edges, premises cited
experiment	compile-verify against an external oracle	`synth.cpp`
J — the jump	*absent*	—

Without J the system could only elaborate what it was given. It could derive `human is_a great_ape` transitively, and it could discover by experiment that the extractor was wrong about a function's arity — but it could not *assert a universal that nothing entailed* and then stake it on an outcome that might destroy it.

---

2. The Jump

2.1 Reading the world, not being told about it

The system must first decide what counts as a function. It is not told. It reads the graph: a node with an outgoing `calls` edge is a function. That is sense experience — a regularity observed in structure that was mechanically extracted from source, with no human labelling step.

```
functions  = { e.from : e ∈ E,  rel(e) = calls }
has_args   = { e.from : e ∈ E,  rel(e) = takes_arg }
```

2.2 Leaping further than the evidence

Counting gives an observed frequency. On the live graph, 55% of nodes identified as functions carry at least one `takes_arg` edge.

The system then asserts:

> **Induced axiom.** *Every function takes at least one argument.*

This is deliberately stronger than the evidence. That gap is the entire point. An induction that merely restated "55% of observed functions have arguments" would be a **summary**, and a summary cannot be refuted — it is already true. Only a universal can be destroyed by a single case, and only something destructible is a scientific claim.

Provenance is recorded as `INDUCED`, the weakest warrant in the system. Its sole justification is that nothing has yet destroyed it.

---

3. Deduction and Experiment

3.1 A -> S

From the induced axiom, a concrete and falsifiable prediction is deduced for a specific symbol:

> If every function takes at least one argument, then calling `run()` with zero arguments must **fail to compile**.

Deduction here is ordinary and mechanical. Nothing interesting happens in this step, which is exactly right: the interesting steps are the leap above it and the experiment below it.

3.2 S -> E: the oracle

The prediction is tested by emitting a translation unit and invoking the C++ compiler:

```
#include "physics.hpp"
int main(){ PhysicsEngine px; px.run(); return 0; }
```

The compiler is the correct oracle for three reasons. It is **deterministic** — the same input yields the same verdict. It is **external** — it knows nothing about the substrate, shares no representation with it, and cannot be argued with. And it is **indifferent** — it has no stake in whether the axiom survives.

An exit code is the refutation.

3.3 The refutation

```
tick            predicted fails       observed compiles    -> REFUTES AXIOM
run             predicted fails       observed compiles    -> REFUTES AXIOM
reset_field     predicted fails       observed compiles    -> REFUTES AXIOM
drive           predicted fails       observed fails       -> CONSISTENT
```

`tick()`, `run()` and `reset_field()` compile with zero arguments. The universal is false. It is retracted, and the retraction is logged to `data/refuted.axioms` — because a failed induction is knowledge: it is a region of hypothesis space now known to be empty.

This is the deliverable. A system that could not be wrong here would not be doing science, and the demonstration that the architecture *can* destroy its own assertion is worth more than any assertion that survives.

---

4. The Cycle Turns: Revision and Re-Jump

Einstein's arrow from experiment back to Axioms does not mean *delete the axiom*. It means **revise it and go round again**. A single disproof is not a cycle.

The revision is itself inductive, and it is constrained. The new axiom must exclude the cases that killed the old one, and must still say something. A revision that excluded each counterexample **by name** would be unfalsifiable bookkeeping — it would predict nothing about a symbol not yet examined. The restriction must therefore be a **structural predicate the graph can evaluate in advance**:

> **Revised axiom.** Every function *for which the graph records at least one* `takes_arg` *edge* takes at least one argument.
>
> **Restriction.** The graph records a `takes_arg` edge for this symbol.

The predicate is observable before any experiment is run, so the revised axiom makes genuine predictions about unseen symbols.

4.1 Testing on held-out symbols

Testing the revision on the symbols that refuted it would be fitting a claim to its own counterexamples. Six symbols not used in round 1 are therefore held out. Symbols falling outside the restriction yield **no prediction** — which is a feature, since a claim that predicts everything predicts nothing.

The revised axiom survives on the held-out symbols that fall inside its scope, and is logged with provenance `induced`.

Survival is not proof. The claim is now harder to destroy, and that is all. Induction confers no warrant beyond not-yet-refuted. What the round demonstrates is **convergence**: a false universal became a restricted claim that reality has not destroyed. Convergence is not guaranteed — that is a property of induction, not a defect of the implementation.

---

5. The Impossibility Register

5.1 The problem

"X cannot be done" is not one kind of claim, and treating it as one has two opposite failure modes. A field can stall for a decade because it mistook a narrow theorem for a general prohibition. A field can also waste a decade chasing something genuinely closed by a conservation law. Both errors come from discarding the *warrant* of the claim and keeping only its content.

The substrate therefore records, for every impossibility claim, **how it is warranted** — because warrant determines what a successful counterexample means.

5.2 Four warrants

Warrant	Basis	What a counterexample does
ASSERTED	someone said so	destroys it; nothing else disturbed
INDUCED	no one has managed it yet	destroys it; the claim was only a survey of failures
DERIVED	follows from a stated axiom	does **not** refute the claim — it refutes the **parent axiom**, and the retraction must propagate upward
THEOREM	follows from a conservation law or proof	is a **scope error until proven otherwise**

The asymmetry between the last two rows is the contribution.

For **DERIVED** claims, the counterexample lands somewhere other than where it appears to. Retracting the impossibility while leaving the axiom that entailed it is incoherent, and leaves every other consequence of that axiom silently unsupported.

For **THEOREM** claims, the default reading of a counterexample is that the new result **left the domain the theorem quantifies over** — not that a proof is broken. Before celebrating, the system must answer: *did this violate the law, or leave its scope?*

5.3 The canonical case

Minsky and Papert proved that a single-layer perceptron cannot compute XOR. The theorem was never violated. Multilayer networks did not break it; they **left its scope**, because the theorem quantifies over single-layer perceptrons and a multilayer network is not one.

Read as a prohibition, the result froze a field. Read correctly, it was a map to the exit: it named exactly the property — single-layer — whose abandonment would be required.

> **A good impossibility theorem does not close a field. It tells you precisely where the bypass must lie.**

5.4 The register, applied to this project's own claims

Self-application is deliberate. A system that records only *other people's* refuted impossibilities is not doing the thing this section describes.

(1) THEOREM — escaped, intact. *"A bag of distinct bytes cannot distinguish* `metformin` *from* `metfromin`*."* Rests on: a multiset is invariant under permutation. The theorem is true and remains true. Bigram lanes are a different representation — ordered *pairs*, not a cleverer weighting of the same multiset — so the sensory layer stopped being the object the theorem quantifies over. Measured 7/7 on misspellings including transpositions. **Scope escape. Not a violation.**

(2) THEOREM — escaped, intact. *"Global viscosity modulation cannot break ground-state locking."* Rests on: uniform scaling preserves all amplitude ratios, and arg-max is invariant under monotone transforms. The rank-preservation proof is untouched and still correct. CSAT escaped by coupling per-cavity through $\tanh(T_i/T_{\text{crit}})$ — it ceased to be *global*. The theorem pointed straight at the fix by naming the property that had to go. **Scope escape. Not a violation.**

(3) THEOREM — stands, not escaped. *"A cognitive architecture can eliminate the* $\log_2 N$ *translation cost."* Rests on: the Translation Capacity Lemma (source coding). This is the honest case. MRS relocates the bits from a label table into a coupling matrix, changing their *organisation* — atomic to factored — and therefore what generalises. **The bit count is unchanged.** Any claim to have removed the cost would be a scope error dressed as a result.

(4) INDUCED — simply refuted. *"Every function takes at least one argument."* Rests on: 55% observed frequency. `tick()` and `reset_field()` compile with zero arguments. An induced universal needs no scope analysis; one counterexample ends it.

5.5 The rule the substrate applies

- **ASSERTED / INDUCED** — one counterexample retracts the claim outright. Nothing else is disturbed.
- **DERIVED** — the counterexample does not touch the claim. It refutes the parent axiom, and retraction propagates to everything else that axiom supports.
- **THEOREM** — a counterexample is a scope error until proven otherwise. Two of the three theorem-claims above were escaped by leaving scope; **none was violated.**

A theorem that is *genuinely* violated means the axioms behind it are wrong. That is the largest event that can occur in this architecture, and it has not occurred.

---

6. Why This Is Not Reinforcement Learning

The distinction is worth stating precisely, because the loop superficially resembles one.

	Reinforcement learning	JAS
Objective	expected return	none
Update	gradient on a value function or policy	retraction of a claim
Signal	scalar reward	accept/reject verdict
Failure	lowers estimated value	**destroys an axiom**
Representation	learned parameters	typed graph edges
What accumulates	a policy	a record of what is false

There is no value function, no policy, no gradient, and no fitted parameter. Nothing estimates a return. A refutation does not adjust a weight — it removes a statement from the system's commitments and logs it as a region of hypothesis space now known to be empty.

The three neuromodulators the substrate carries (dopamine, cortisol, serotonin) do move with outcome, but they are scalar multipliers on physical constants — drive, damping, and Hebbian rate — with homeostatic pull back to baseline. A run of refutations raises cortisol and therefore damping, so the field literally propagates less far after failure. This is affect, not credit assignment: nothing estimates expected future reward, and no action is selected to maximise anything.

---

7. Falsifiability of This Paper's Own Claims

Claim	How to destroy it
The jump is not entailed by the evidence	exhibit an inference rule deriving the universal from 55% frequency
Refutation is external	show the oracle shares state with the substrate
The revision is not fitted to its counterexamples	show a held-out symbol inside the restriction that refutes it
Theorem-claims (1) and (2) were escapes, not violations	exhibit a byte **multiset** distinguishing the anagram pair, or a **globally** uniform modulation breaking ground-state locking
Claim (3) stands	exhibit an architecture discriminating N outcomes with $< \log_2 N$ bits anywhere in the system

Each row is a concrete experiment, and the last two are the ones that would matter most.

---

8. Limitations

One induction schema. The system currently jumps over a single relation (`takes_arg`) with a single generalisation form. A general mechanism would enumerate candidate universals across relation types and rank them by refutability. Nothing in the architecture forbids this; it is unwritten.

Revision is hand-shaped. The restriction predicate is chosen by the implementer, not searched for. An automatic version would need a principled bias toward restrictions that remain predictive on unseen symbols — and that bias is itself an inductive assumption which the register would then have to record.

The register is curated. Its four entries were entered by a human who noticed them. A system that detected its own impossibility claims and classified their warrant automatically would be a stronger result, and does not exist.

One oracle. The compiler settles questions about compilability and nothing else. Domains without a cheap deterministic oracle — most empirical science — do not inherit this loop for free.

---

9. Conclusion

The J-A-S cycle is implementable, and implementing it costs almost nothing: induction is counting, deduction is graph traversal, and refutation is an exit code. What it buys is a system that can hold a claim, stake it on an outcome, and lose.

The impossibility register is the more portable contribution. Recording the *warrant* of a "cannot" — rather than only its content — turns impossibility results from prohibitions into maps. Under this discipline the question after an apparent breakthrough is not *"is the theorem broken?"* but *"which quantified property did we abandon?"* — and in every case examined here, including two of the project's own, the answer named the exit precisely.

The corresponding discipline for a claim that a system *can* do something is the one this lineage learned the hard way: re-derive it from the running code before repeating it.

References

- Einstein, A. Letter to Maurice Solovine, 7 May 1952. (The J-A-S diagram.)
- Minsky, M. and Papert, S. *Perceptrons*, MIT Press, 1969.
- Popper, K. *The Logic of Scientific Discovery*, 1959.
- Mahir, I. *From Artificial to Synthetic Intelligence: Principles of Constructive Structural Cognition.* §5.4c (experiment provenance), §5.4e (the cycle), §6.2 (lineage audit).
- Mahir, I. *Constructive Language Locking (CLL).* `cll.md` — the category-error argument whose refutation pattern §5 generalises.
- Implementation: `src/jas.cpp` (cycle and register), `src/synth.cpp` (compile-verify oracle), `data/refuted.axioms` (refutation log).